

CC3092 - Deep Learning y Sistemas Inteligentes

Laboratorio #1

Entrenamiento de Redes Neuronales

Instrucciones generales

- Individual.
- Entrega: domingo 2 de agosto, 2026. 23:59.
- Trabajar en Python, en un Jupyter Notebook (.ipynb).
- Framework: PyTorch. Instale también pandas, numpy, matplotlib, scikit-learn (para el split y las métricas).

1. Dataset

Trabjarán con el dataset público "California Housing Prices", un problema clásico de regresión donde se predice el valor mediano de una vivienda (median_house_value) a partir de características socioeconómicas y geográficas de distritos censales de California.

Enlace de descarga: <https://www.kaggle.com/datasets/camnugent/california-housing-prices>

Dataset en sklearn: `from sklearn.datasets import fetch_california_housing`

Pueden usar cualquiera de las dos fuentes.

2. Exploración y preparación de los datos

Antes de construir el modelo, cargue el dataset y responda dentro del notebook (en celdas de texto):

- ¿Cuántas observaciones y cuántas variables tiene el dataset?
- ¿Qué representa cada variable (feature) y cuál es la variable objetivo (target)?
- ¿Hay valores nulos, duplicados o atípicos (outliers)? ¿Cómo los trató?
- ¿Qué variables son numéricas y cuáles categóricas? ¿Cómo codificó las categóricas?
- ¿Fue necesario normalizar o escalar las variables numéricas?

Divida el dataset en conjuntos de entrenamiento, validación y prueba, respetando la regla vista en clase: el conjunto de test no debe influir en ninguna decisión de entrenamiento ni de ajuste de hiperparámetros.

3. Investigación: optimizadores y capas de PyTorch para el MLP

Investigue y documente en el notebook las capas de torch.nn necesarias para construir un MLP de regresión. Para cada capa, describa brevemente su propósito y sus parámetros más relevantes. Como mínimo debe investigar:

- nn.Linear.
- Funciones de activación: nn.ReLU, nn.LeakyReLU, nn.Tanh
- nn.Dropout.
- nn.BatchNorm1d.
- Funciones de pérdida para regresión: nn.MSELoss, nn.L1Loss, nn.SmoothL1Loss.
- Optimizadores en torch.optim: SGD, Adam, RMSprop. Documente el rol del parámetro lr (learning rate) y, si aplica, weight_decay (regularización L2). Para cada uno de estos optimizadores, realice una

breve investigación sobre cuál es su función en el entrenamiento de un MLP y qué características lo diferencian del resto.

4. Entrenamiento e iteración de hiperparámetros

Entrene su MLP y realice al menos 10 iteraciones del modelo, cambiando sistemáticamente uno o más de los siguientes elementos entre iteraciones:

- Arquitectura: número de capas ocultas y número de neuronas por capa.
- Función(es) de activación.
- Optimizador y learning rate.
- Batch size y número de epochs.
- Estrategia de regularización: L1, L2 (weight_decay), dropout, o combinaciones.

No es necesario que las 10 iteraciones sean completamente aleatorias: se recomienda seguir una estrategia de búsqueda de hiperparámetros (grid search o random search, vistos en clase) para explorar el espacio de forma más ordenada, cambiando idealmente una o dos variables a la vez para poder atribuir el efecto de cada cambio.

Para cada iteración, registre:

1. La configuración de hiperparámetros usada
2. La pérdida (loss) de entrenamiento y de validación por epoch.
3. Las métricas de evaluación del problema de regresión: MSE, MAE, RMSE, calculadas sobre el conjunto de validación.

Grafique las curvas de pérdida de entrenamiento y validación de al menos 3 de sus iteraciones, para poder identificar visualmente señales de overfitting o underfitting.

Una vez identificada la mejor configuración según sus métricas de validación, evalúe ese modelo final una única vez sobre el conjunto de test y reporte sus métricas.

5. Documentación de resultados

Presente, en el notebook y en el reporte, una tabla resumen con las 10 (o más) iteraciones realizadas. Ejemplo:

#	Arquitectura	Activación	Optimizador / LR	Batch / Epochs	Regularización	MSE (val)	MAE (val)	RMSE (val)
1	[64, 32]	ReLU	Adam / 0.001	32 / 50	—	...	...	...
...	...	...	...	...	...	...	...	...

Esta tabla es solo una guía de formato; pueden agregar columnas adicionales si lo consideran útil.

6. Discusión y análisis

En el reporte escrito, responda con base en sus resultados:

- ¿Qué cambio de hiperparámetro tuvo el mayor impacto positivo en las métricas de validación? ¿Y el mayor impacto negativo?
- ¿Observó overfitting o underfitting en alguna de sus iteraciones? ¿Cómo lo identificó y qué hizo para mitigarlo?
- ¿La regularización (L1, L2 o dropout) mejoró el desempeño en validación? ¿Cuál funcionó mejor para este dataset y por qué cree que fue así?
- ¿Cómo se relacionan el batch size y el número de epochs con la estabilidad y velocidad del entrenamiento que observó?
- Comparando MSE, MAE y RMSE, ¿qué le dice cada métrica sobre el comportamiento de su modelo que las otras no muestran?
- Si tuviera que entrenar un modelo de producción con este dataset, ¿qué arquitectura e hiperparámetros elegiría, y qué estrategia de búsqueda (grid, random o bayesiana) usaría para seguir optimizando? Justifique.

Entregables

Entregable	Contenido
PDF (máx. 3 páginas)	Investigación de optimizadores, breve descripción de los parámetros más importantes de cada capa investigada, tabla de resultados de las iteraciones, análisis de resultados y conclusiones.
Repositorio (Git)	Jupyter Notebook completo y comentado: carga y preparación de datos, investigación de capas, definición del modelo, entrenamiento de las 10+ iteraciones y evaluación final sobre test.

Incluya el enlace al repositorio al inicio del PDF.